

Доклад по лабораторной работе №8

Шифр гаммирования

Архипов Александр Сергеевич НБИбд-01-24 1132239106 |
1132239106@rudn.ru

07.10.2026

Цели и задачи

Цель лабораторной работы

Освоить на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом.

Выполнение лабораторной работы

Гаммирование

Гаммирование – это наложение (снятие) на открытые (зашифрованные) данные криптографической гаммы, т.е. последовательности элементов данных, вырабатываемых с помощью некоторого криптографического алгоритма, для получения зашифрованных (открытых) данных.

Алгоритм взлома

Шифротексты обеих телеграмм можно получить по формулам режима однократного гаммирования:

$$C_1 = P_1 \oplus K$$

$$C_2 = P_2 \oplus K$$

Алгоритм взлома

Открытый текст можно найти, зная шифротекст двух телеграмм, зашифрованных одним ключом. Для это оба равенства складываются по модулю 2. Тогда с учётом свойства операции XOR получаем:

$$C_1 \oplus C_2 = P_1 \oplus K \oplus P_2 \oplus K = P_1 \oplus P_2$$

Алгоритм взлома

Предположим, что одна из телеграмм является шаблоном — т.е. имеет текст фиксированный формат, в который вписываются значения полей. Допустим, что злоумышленнику этот формат известен. Тогда он получает достаточно много пар $C_1 \oplus C_2$ (известен вид обеих шифровок). Тогда зная P_1 имеем:

$$C_1 \oplus C_2 \oplus P_1 = P_1 \oplus P_2 \oplus P_1 = P_2$$

Схема работы алгоритма

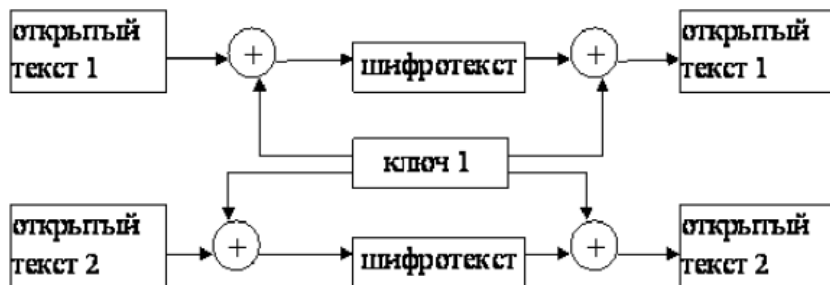


Рис. 1: Работа алгоритма гаммирования

Пример работы программы

```
Ариков Александр Сергеевич | НБ60д-01-24
lpr8 / 01: атака на повторно использованный XOR-ключ
guest@arkhipov-lab:~/crypto$ cd /home/guest/crypto
sed -n '1,1ip' lab80.py
python3 lab80.py attack
import secrets
import sys
def xor(a, b):
    if len(a) != len(b):
        raise ValueError("Длины байтовых строк должны совпадать")
    return bytes(x ^ y for x, y in zip(a, b))
p1 = "Кодоваяфраза1".encode('utf-8')
p2 = "Безопасность2".encode('utf-8')
assert len(p1) == len(p2)
key = secrets.token_bytes(len(p1))
c1, c2 = xor(p1, key), xor(p2, key)
Ариков Александр Сергеевич | НБ60д-01-24
P1: Кодоваяфраза1
P2: Безопасность2
Длина в байтах: 25
C1 (hex): b8ab9945ef391c679af2ae2c4efb0b4b57fedb66f85680da49
C2 (hex): b8a0994def3a1c679affae2c4ef58b5256c0da57f96381e64a
Восстановленный P2 из C1, C2 и известного P1: Ок
Восстановление ключа по известному P1: Ок
guest@arkhipov-lab:~/crypto$
guest@arkhipov-lab:~/crypto$
```

Рис. 2: атака на повторно использованный XOR-ключ

Шифрование и восстановление

```
Аркадий Александр Сергеевич | НЕМД-01-24
lpr / #2: точное XOR-шифрование и расшифровка UTF-8
guest@arkhipov-lab:~/crypto$ cd /home/guest/crypto
nl -ba lab88.py
python3 lab88.py
1 import secrets
2 import sys
3 def xor(a, b):
4     if len(a) != len(b):
5         raise ValueError("Длины байтовых строк должны совпадать")
6     return bytes(x ^ y for x, y in zip(a, b))
7 p1 = "Кодовыйфрагм1.encode('utf-8')
8 p2 = "Безопасность2.encode('utf-8')
9 assert len(p1) == len(p2)
10 key = secrets.token_bytes(len(p1))
11 c1, c2 = xor(p1, key), xor(p2, key)
12 print('Аркадий Александр Сергеевич | НЕМД-01-24')
13 print('P1:', p1.decode())
14 print('P2:', p2.decode())
15 print('Длина = байтах:', len(p1))
16 print('C1 (hex):', c1.hex())
17 print('C2 (hex):', c2.hex())
18 if len(sys.argv) > 1 and sys.argv[1] == 'attack':
19     recovered_p2 = xor(c1, p1)
20     recovered_p2 = xor(xor(c1, c2), p1)
21     assert recovered_key == key
22     assert recovered_p2 == p2
23     print('Восстановленный P2 из C1, C2 и известного P1:', recovered_p2.decode())
24     print('Восстановление ключа по известному P1: OK')
25 else:
26     d1, d2 = xor(c1, key), xor(c2, key)
27     assert d1 == p1 and d2 == p2
28     print('Расшифрованный P1:', d1.decode())
29     print('Расшифрованный P2:', d2.decode())
30     print('Точное восстановление обоих сообщений: OK')
Аркадий Александр Сергеевич | НЕМД-01-24
P1: Кодовыйфрагм1
P2: Безопасность2
Длина в байтах: 25
C1 (hex): 8a5c4066e143630c9784c90b0556a1d5ba97d3b6aca7fb455dd
C2 (hex): 8a5c4066e143630c9784c90b0556a1d42a8433a5bcb4ab5669de
Расшифрованный P1: Кодовыйфрагм1
Расшифрованный P2: Безопасность2
Точное восстановление обоих сообщений: OK
guest@arkhipov-lab:~/crypto$
guest@arkhipov-lab:~/crypto$ █
```

Рис. 3: точное XOR-шифрование и расшифровка UTF-8

Среда выполнения

Программа запущена как Python-файл в настоящем терминале Ubuntu, без Jupyter. Используется XOR байтов UTF-8 и ключ `secrets.token_bytes`. Обе строки имеют длину 25 байт. Повторение ключа в двух сообщениях используется только в учебном примере атаки. Шифротексты меняются при каждом новом запуске.

Выводы

Результаты выполнения лабораторной работы

В ходе выполнения лабораторной работы было разработано приложение, позволяющее шифровать тексты в режиме однократного гаммирования.